

Manual de Usuario y Documentación de Uso

1. Introducción

Este proyecto es una aplicación que consta de dos microservicios desarrollados en Java y empaquetada usando Docker, lo que permite su compilación y ejecución sin depender de tener Java instalado en el sistema host. El uso de contenedores permite una implementación más sencilla y consistente en cualquier ambiente de producción.

Objetivos principales:

- **Compilación dentro de Docker:** Los proyectos se compila en un contenedor, usando una imagen de Maven que incluye Java, lo que permite generar el archivo JAR de manera independiente.
 - **Despliegue en producción:** Se utilizan dos contenedores Docker basado en una imagen ligera (openjdk:11-jre-slim) para ejecutar la aplicación de manera óptima.
 - **Facilidad de integración:** Se incluye documentación para su versión en Git y para la gestión del ciclo de vida del proyecto.
-

2. Requisitos Previos

Antes de poner en marcha el sistema, asegúrate de contar con lo siguiente:

- **Docker:** Instalado y configurado. Consulta la documentación oficial de Docker para su instalación según tu sistema operativo.
 - **Git:** Para el control de versiones y para poder actualizar el código o mantener un historial de cambios.
 - **Acceso a la terminal o consola de comandos:** Para la ejecución de scripts y comandos de Docker.
-

3. Preparación del Proyecto

3.1. Desempaquetado del Proyecto

Si has descargado el proyecto en un archivo ZIP desde el repositorio (<https://github.com/rosaguvi/PruebaTecnica>):

1. Ubica el archivo ZIP en tu máquina.
2. Abre una terminal y navega hasta el directorio donde se encuentra el archivo.
3. Ejecuta el siguiente comando para descomprimirlo:

unzip workspace.zip

4. Ingresa al directorio descomprimido:

```
cd nombre_del_directorio_extraido
```

Nota: Asegúrate de que en la raíz de la carpeta se encuentren archivos como “run_all.bat” o “run_all.sh”, los cuales se encargan de desplegar las aplicaciones en los contenedores de Docker.

4. Despliegue de la aplicación

4.1 Cómo ejecutar en local (Windows)

1. Asegúrate de tener Docker Desktop instalado y en ejecución.
2. Abre una terminal (CMD o PowerShell) en la carpeta del proyecto.
3. Ejecuta:

```
run_all.bat
```

Esto compilará los servicios y los ejecutará en:

- Autenticación: <http://localhost:8081/swagger-ui.html>
- Conversión Divisas: <http://localhost:8082/swagger-ui.html>

4.2. Cómo ejecutar en Linux/Mac

```
chmod +x run_all.sh
```

```
./run_all.sh
```

El proceso de compilación y empaquetado se realiza enteramente dentro de Docker. Esto permite generar el JAR y preparar la imagen final sin necesidad de contar con Java de forma local.

4.3. Acceso a la Aplicación

Una vez que los contenedores estén corriendo, accede a la aplicación a través de tu navegador web en la siguiente dirección:

<http://localhost:8081/auth/login> y <http://localhost:8082/api/convertir>

Recomendación: Verifica el puerto configurado en el Dockerfile o en la configuración de la aplicación en caso de que sea diferente.

4.4. Monitoreo y Logs

Para visualizar los logs del contenedor, puedes usar:

```
docker logs servicio-autenticacion/servicio-conversion-divisas
```

Esto permitirá identificar mensajes, errores o información relevante durante la ejecución.

5. Uso del Sistema

5.1. Funcionalidades Principales

- **Inicio y autenticación:** la aplicación cuenta con un módulo de autenticación, la cual se puede acceder desde Postman o una aplicación similar, para más de talles acceder <http://localhost:8081/swagger-ui/index.html>

```
{  
  "username": "usuario",  
  "password": "clavedeusuario"  
}
```

- **Interfaz de usuario o API:** La cuenta con la funcionalidad de conversión de divisas, la cual se accede desde postman, y se debe proveer el token generado por el módulo de autenticación, para más de talles acceder <http://localhost:8082/swagger-ui/index.html>

```
{  
  "monto": 100.0,  
  "monedaOrigen": "COP",  
  "monedaDestino": "USD"  
}
```

6. Mantenimiento y Actualización

6.1. Actualización del Código Fuente

Cuando se realicen cambios en el código fuente del proyecto:

1. Actualiza el repositorio y realiza los cambios necesarios.
2. Vuelve a ejecutar el .bat o .sh para la actualización del sistema.

6.2. Uso de Git para el Control de Versiones

Si aún no lo has hecho, inicializa el repositorio Git en la raíz del proyecto. Se recomienda seguir estos pasos:

```
git init
```

```
git add .
```

```
git commit -m "Actualización: integración de Dockerfile y documentación de uso"
```

```
git branch -M main
```

```
git remote add origin https://github.com/tu_usuario/nombre_repositorio.git
```

```
git push -u origin main
```

Esto permitirá mantener un historial de cambios, revertir actualizaciones si es necesario y facilitar la colaboración.

7. Solución de Problemas y FAQ

7.1. Problemas Comunes

- **Error en la compilación:**
 - Verifica que el archivo pom.xml esté correctamente configurado.
 - Revisa la salida del comando docker build para identificar errores específicos.
- **La aplicación no arranca en el contenedor:**
 - Usa docker logs *servicio-autenticacion/ servicio-conversion-divisas* para revisar la salida y detectar posibles errores en tiempo de ejecución.
 - Asegúrate de que el puerto expuesto en el Dockerfile coincide con el mapeo realizado en el comando docker run.
- **Problemas de red o acceso:**
 - Confirma que el puerto configurado no está siendo utilizado por otra aplicación en el host.
 - Revisa la configuración del firewall o políticas de red en el host.

7.2. Preguntas Frecuentes (FAQ)

1. **¿Puedo ejecutar pruebas durante la compilación?**

Sí, elimina la opción `-DskipTests` en el comando `mvn clean package` dentro del Dockerfile para incluir la ejecución de pruebas.

2. **¿Cómo actualizo la configuración del puerto?**

Modifica el valor del comando `EXPOSE` en el Dockerfile y actualiza el comando `docker run -p` según el puerto deseado.

3. **¿Cómo accedo a la shell del contenedor?**

Puedes acceder al shell del contenedor usando el siguiente comando:

```
docker exec -it servicio-autenticacion/ servicio-conversion-divisas sh
```

8. Consideraciones Finales

- **Actualizaciones:** Se recomienda revisar periódicamente la documentación y la configuración del Dockerfile para incorporar nuevas mejoras o requerimientos de seguridad.
- **Ambiente de pruebas:** Antes de pasar a producción, ejecuta pruebas en un ambiente controlado para garantizar el correcto funcionamiento del sistema.
- **Seguridad:** Valida y actualiza las dependencias regularmente para mantener la seguridad en el entorno de producción.